Defense Notes

Deliverable D8 support: the live demo script (spec 11.1), answers to the defense questions (spec 11.2), and the points to make about architecture, security, testing and limitations. Record names and numbers refer to the demo dataset as seeded (`database/seeds/README.md`); amounts and dates depend on the day it was seeded.

---

1. Before the defense

The day before (or the morning of):

1. Reseed a fresh demo database so the months are current: `CREATE DATABASE bcis_demo OWNER bcis_app;`, point `DATABASE_URL` at it, then `pnpm --filter ./apps/api db:migrate` and `pnpm --filter ./apps/api db:seed-all`.
2. On the server PC: start the API (the scheduled task, or `pnpm --filter ./apps/api start` after a build). Check `http://<server>:3000/health/db` answers.
3. On each client PC: open BCIS Billing, check the server address under the sign-in box.
4. Take a backup (Backup & Restore > Back up now) so there is a fresh restore point, and run the integrity check once (it should pass).
5. Have open: the acceptance test report PDF, the technical documentation PDF, and a PowerShell window for step 11.
6. Know the passwords: the owner (from `.env`) and `SEED_DEMO_PASSWORD` for the demo users.

If something fails during the demo: stay calm, read the message aloud (it says what is wrong), and show the same rule in the acceptance test report instead.

---

2. Live demo script (spec 11.1)

#	Spec step	Do this	Point out
1	Log in as administrator, show the dashboard	Sign in as `demo_admin`.	KPIs come from the same tables as the reports; aging and collection rate; collectors' shortages. Charts have a Table view.
2	Create or open a subscriber with an Internet/Cable service	Subscribers > New Subscriber: name, billing day (e.g. 28), address, mobile `0999 000 9999`. Create. On **Services**, Add service: **INET-25 Internet 25 Mbps (₱999)**. Open it and set status **Active** with today's date.	Account number assigned by the server; status change needs a reason and is audited (Audit tab).
3	Generate or display the monthly invoice and ledger debit	Billing > Generate Billing, this month: **Generate drafts** (the new service appears), **Finalize**. Back on the subscriber: **Billing** tab shows the ₱999 invoice; **Ledger** tab shows the debit.	Two steps: drafts have no number; finalize numbers them under a lock and posts the ledger. Re-running creates nothing twice (AT-11).

#	Spec step	Do this	Point out
4	Post an exact or partial cash payment, print/preview the receipt	Payments > Receive Payment, the new subscriber, amount **999.00**, cash, **Post payment**. Print the receipt (print preview).	The allocation preview, receipt number, invoice now **Paid**, ledger balance ₱0 (AT-01). For partial: post 500 on another subscriber and show **Partially paid** (AT-02).
5	Post/verify a GCash payment with proof, show duplicate-reference protection	Payments > GCash Verification, Pending: the submission from **Karlo Nacua (BCIS-000004)**. Show the proof image. **Verify...** (posts the payment). Then **Record GCash payment** for any subscriber with the **same reference** → refused, naming the receipt.	Reference unique in the database; proof required before verify (AT-05).
6	Show the subscriber ledger and Statement of Account	Open **BCIS-000017 Manny Bacani** (two services, seven open bills). Ledger tab; Documents tab > **Export PDF**.	Running balance; statement says it reconciles (open invoices − credit = ledger).
7	Open an overdue subscriber and demonstrate AR aging/filtering	Receivables > Aging (all buckets), then Overdue; filter by bucket **90+** or by collector; open **BCIS-000042 Rosa Magbanua** (oldest bill, service suspended).	Aging total = open invoice balances; credit shown separately; the reconciliation test proves it on the whole dataset.
8	Create/open a collector batch, record collections, remit cash and reconcile	Collections > Collection Batches > **CB-000005** (in progress). Record collection for one subscriber on it (cash). **Submit...**, **Record remittance** for ₱100 less than collected, **Reconcile...** (shortage shown; reason required), **Close batch...** (must confirm the same difference). Also open **CB-000004** (closed with a ₱500 shortage).	The server computes the difference; a short batch cannot be closed as balanced (AT-07, AT-08).
9	Demonstrate a payment reversal and its audit trail	Payment History > open the payment from step 4 > **Reverse payment**, reason "Demo: posted to the wrong subscriber". Show the invoice open again, the ledger with both entries, then Administration > Audit Log filtered to payments. Also show **RCPT-000070** (reversed in the seed: an extra zero keyed).	Nothing deleted; receipt number kept; actor and reason recorded (AT-06).
10	Show a management report exported to PDF/XLSX	Reports > Collections (last month) > **Export PDF** and **Export Excel**; open both.	Same figures in both formats; exports are audited.
11	Demonstrate role restrictions with a lower-privileged account	Sign out; sign in as `demo_cashier`: smaller menu. Then show the server refusing a direct call (PowerShell below).	Hiding menus is cosmetic; the server checks permissions on every request (AT-10).
12	Show backup creation and explain the tested restore	Sign in as the owner. Administration > Backup & Restore > **Back up now** (verified). Press **Test restore** on it (restores into `bcis_verify`, integrity passes). Explain Restore... (typed number, safety backup, maintenance mode, integrity check after).	Backups verified by checksums and pg_restore; AT-12 restores for real in the test suite.

#	Spec step	Do this	Point out
13	Simultaneous operation from several clients	Two PCs: both open Receive Payment and post at the same moment; show consecutive receipt numbers. Or run `pnpm --filter ./apps/api concurrency` against the server.	One API, transactions and row locks; numbering locked until commit (AT-09).

Direct API call for step 11 (PowerShell on any PC; replace the server address):

```
$server = "http://192.168.1.10:3000"
$password = Read-Host "demo_cashier password"
$login = Invoke-RestMethod -Method Post -Uri "$server/auth/login" -ContentType "application/json" `
  -Body (@{ username = "demo_cashier"; password = $password } | ConvertTo-Json)
Invoke-RestMethod -Method Post -Uri "$server/backups" -ContentType "application/json" `
  -Headers @{ Authorization = "Bearer $($login.token)" } -Body "{}"
# -> 403 Forbidden: "You do not have permission to perform this action."
```

---

3. Defense questions (spec 11.2)

Why PostgreSQL instead of a shared SQLite database for three PCs? A database file on a shared folder relies on network file locking, which is unreliable, and gives every PC direct write access with no one enforcing permissions. Here one API owns the database: PostgreSQL's transactions and row locks make concurrent posting safe, constraints and triggers enforce rules whatever the client does, permissions are checked on the server, and `pg_dump` takes consistent backups while people work. (Technical documentation 2.2.)

Where does authoritative billing/payment logic live, and why? In the API's service layer (`apps/api/src/billing`, `payments`, `collection`) and the shared rules package (`packages/shared`: allocation, money, aging), inside database transactions; key rules are repeated as database constraints and triggers. The desktop app only displays and submits. One authority means every PC gets the same result, a modified client cannot bypass a rule, and the rules are testable without a UI.

How do you prevent duplicate invoices, duplicate receipts and duplicate GCash transactions?

- Invoices: generation only creates drafts for accounts with no live invoice that month, under a lock on the billing month; the unique index `invoices_one_per_period_idx` (account, month, not void) makes a second one impossible (AT-11).
- Receipts and all document numbers: taken from `document_sequences` inside the posting transaction; the counter row stays locked until commit, so numbers are consecutive and never repeat, and a failed posting gives its number back (AT-09). Unique constraints on the numbers too.
- GCash: references are normalized and unique in the database; a second submission is refused with the receipt that used it (AT-05).

How is a partial payment represented in the database? A `payments` row with the full amount, and a `payment_allocations` row for the part applied to the invoice. The invoice's `paid_centavos` goes up by that amount; its status becomes `partially_paid`; its balance is `total + adjusted - paid`. The ledger has the invoice's debit and the payment's credit, so the subscriber's balance is the difference (AT-02).

Why are posted payments reversed instead of deleted? Because the money really was recorded: a receipt was printed, a ledger entry exists, reports were run. Deleting would make history and printed documents disagree and hide mistakes or fraud. A reversal adds a linked record and a ledger debit, reopens the invoices, keeps the receipt number, and records who and why. Database triggers back this up: a payment cannot be deleted, a posted payment's amount, date, method or subscriber cannot be edited, a reversed one cannot change at all, and allocations, reversals and ledger entries are append-only (AT-06).

How do you prove the ledger balance is correct? Three ways. (1) The identity: for every subscriber, the ledger sum equals open invoice balances minus unapplied credit; the Statement of Account checks it and says so. (2) The integrity check verifies paid amounts equal allocations, allocations never exceed payments, and the ledger reconciles, across the whole database (on demand, after every test restore and restore). (3) The reconciliation test seeds four months of activity and checks every report against the records (acceptance report, "Reconciliation checks").

What happens if the API crashes halfway through a payment posting? The posting is one transaction: payment, allocations, invoice updates, ledger entry, receipt number, audit row. Until `COMMIT`, PostgreSQL holds none of it; if the process dies, the transaction is rolled back entirely. No half payment and no used receipt number. The API restarts automatically (watchdog task). The cashier sees an error and checks Payment History before posting again, in case the commit happened just before the connection dropped.

How does server-side authorization differ from hiding buttons in the UI? Hiding a button only changes what the screen shows; anyone can call the API directly. The server checks the session and the permission on every request, reading the user's roles from the database (`authenticate` + `requirePermission`). A test calls every one of the 121 routes without a session and as a user with no roles, and expects 401 and 403 (`security/route-guards.test.ts`). Demo step 11 shows it live.

How are backups validated and restored? Each backup is a `pg_dump` of one consistent snapshot plus the proof images and a manifest with row counts and SHA-256 checksums. Verification re-checks every checksum and that `pg_restore` can read the dump. Test restore loads it into a separate database and runs the integrity check, without touching live data. A real restore requires the owner, the typed backup number and a reason; it takes a safety backup first, blocks other users, restores in a single transaction, signs everyone out and runs the integrity check (AT-12).

What design changes would a future collector mobile/PWA application need? Collector accounts limited to their own batch; the API reachable from outside the office (TLS, rate limiting); offline storage of collections with a client-generated id per collection so a retried upload cannot post twice (idempotency); receipts numbered by the server on sync (or a series per collector); and rules for conflicts, for example when the office posts a payment for the same subscriber before the collector syncs. The batch, reconciliation and allocation rules already live on the server and would be reused unchanged. (Technical documentation 14.)

---

4. Points to make

- **Architecture**: three Electron clients, one Fastify API, PostgreSQL; the renderer never touches the database; the session token stays in Electron's main process; one IPC handler per API call.
- **Financial integrity**: integer centavos; append-only ledger and history (triggers); reversals, voids and adjustments instead of edits; numbering under locks; constraints as a safety net.

- **Security**: argon2id; hashed session tokens with idle and absolute timeouts; lockout; forced password change; permission on every route, proven by a test over the whole route table; strict Electron settings; audit of every change.
- **Testing**: 770 automated tests against a real database; AT-01..AT-12 with the spec's own data over HTTP; reconciliation of every report; concurrency tests; a bug log with root causes.
- **Limitations and future work**: plain HTTP on the LAN (TLS needed before any outside access); no pro-rated billing; no SMS reminders or customer portal; collector app (above).
- **AI assistance**: Claude Code was used under the rules in `CLAUDE.md`, in small reviewed steps with tests; be ready to open any file and explain it.